COS30049 – Computing Technology Innovation Project

Week 12 - Software Testing

Ningran Li
Ningran.li@swin.edu.au

Acknowledgement of Country

We respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

We are honoured to recognise our connection to Wurundjeri Country, history, culture, and spirituality through these locations, and strive to ensure that we operate in a manner that respects and honours the Elders and Ancestors of these lands.

We also respectfully acknowledge Swinburne's Aboriginal and Torres Strait Islander staff, students, alumni, partners and visitors.

We also acknowledge and respect the Traditional Owners of lands across Australia, their Elders, Ancestors, cultures, and heritage, and recognise the continuing sovereignties of all Aboriginal and Torres Strait Islander Nations.



- Introduction to Software Testing
- Types of Software Testing
- Test Planning and Strategy
- **Test Case Design Techniques**
- Test Execution and Reporting
- Automated Testing

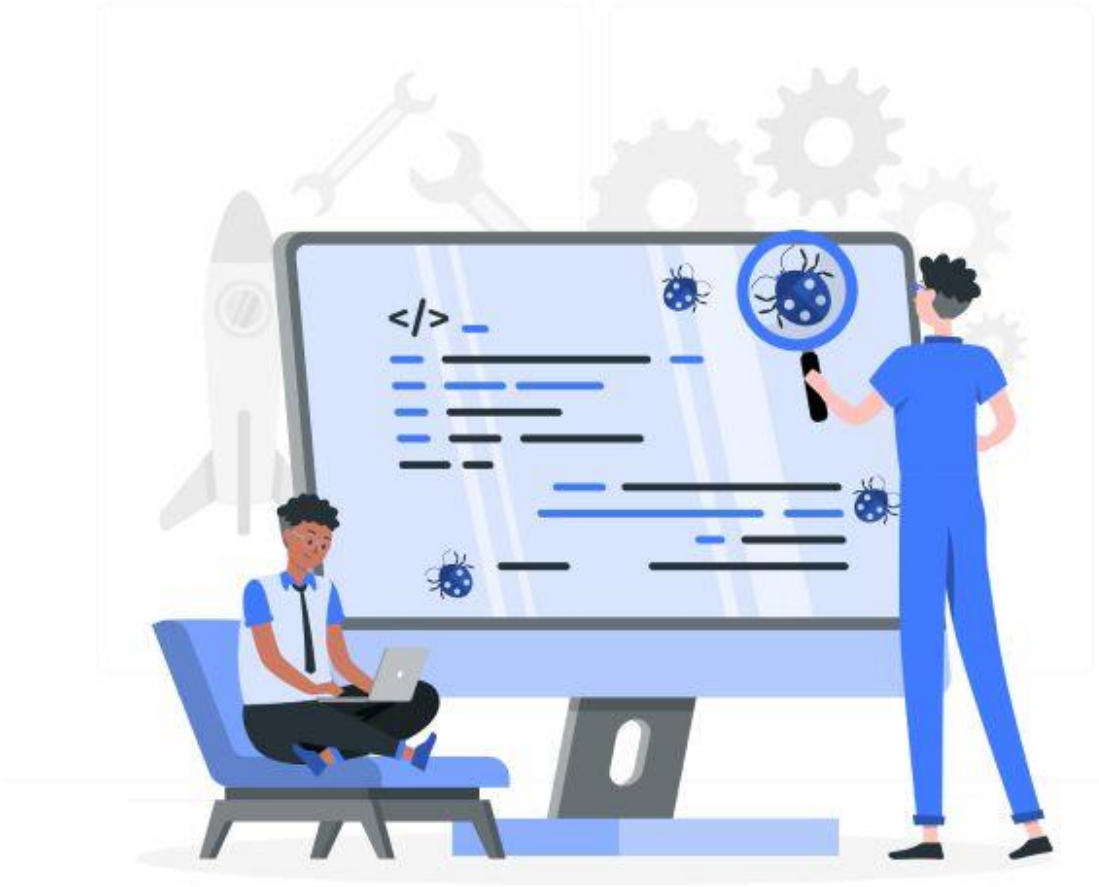


Image from [Pinterest]

Introduction to Test Case Design Techniques

Definition :

Systematic methods for creating test cases to effectively identify defects

Importance:

Ensures comprehensive coverage of software functionality

Maximizes defect detection with minimal test cases

Provides a structured approach to testing

Categories of techniques:

Black-box (specification-based) | White-box (structure-based)

Experience-based techniques

Best practices:

Combine multiple techniques for better coverage

Tailor techniques to specific project needs

Maintain traceability between requirements and test cases



Image from [Pinterest]

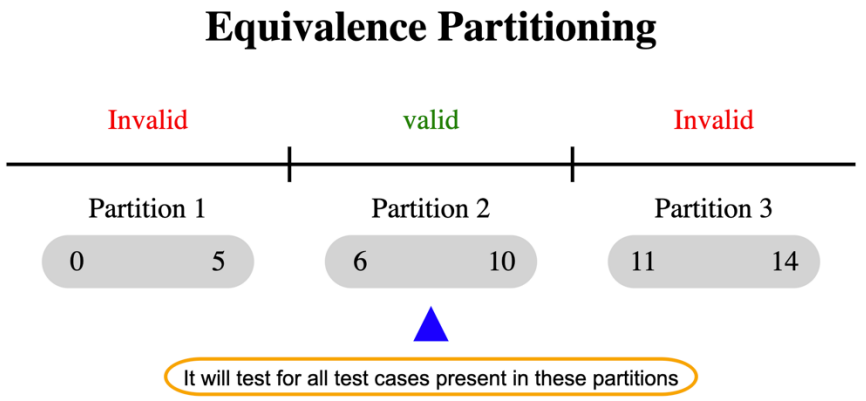
Equivalence Partitioning

Definition :

Dividing input data into partitions of equivalent data from which test cases can be derived

Steps to apply:

1. Identify the input parameters or conditions
2. Determine valid and invalid partitions for each parameter
3. Create test cases for each partition
4. Select representative values from each partition



Benefits:

Reduces the number of test cases without compromising test coverage

Helps in identifying boundary conditions

Example : Testing a login form

Username: Valid (3-20 characters), Invalid (empty, >20 characters)

Password: Valid (8-16 characters, with special chars), Invalid (too short, no special chars)

Boundary Value Analysis (BVA)

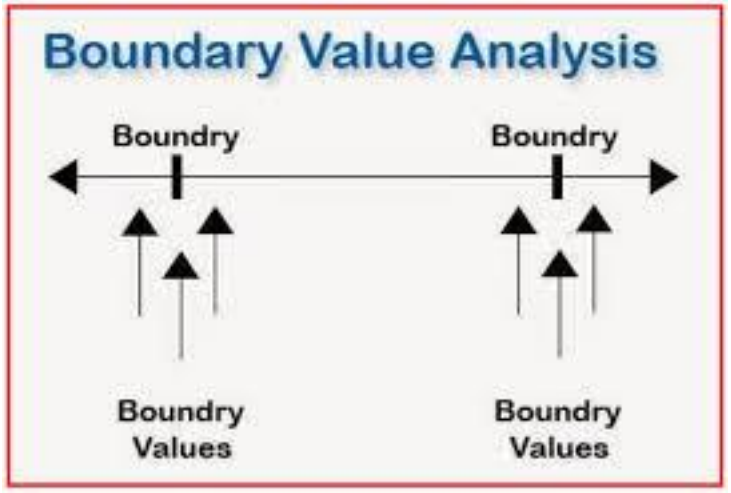


Image from [Source]

Definition :

Testing at the boundaries between partitions

Steps to apply:

1. Identify the boundaries for each input parameter
2. Create test cases for the boundary values (just inside and outside the boundaries)

Common boundary scenarios:

Minimum and maximum values | Just inside and outside valid ranges

First and last elements of a list | Empty or full states

Benefits:

High defect detection rate at boundaries | Focuses on critical areas where errors are likely

Example : Testing age restrictions for a movie ticket system

Boundary values: 0, 1, 12, 13, 17, 18, 65, 66, 120, 121

Test cases: Age = 0 (invalid), 1 (child), 12 (child), 13 (teen), 17 (teen), 18 (adult), 65 (adult), 66 (senior), 120 (senior), 121 (invalid)

Decision Table Testing

Definition :

A systematic approach to test system behaviour for different input combinations

Components:

Conditions | Actions | Rules

Steps to apply:

1. Identify all conditions and possible actions.
2. Determine all possible combinations, and fill in the actions for each combination
3. Simplify the table by combining similar rules if possible

Benefits:

Ensures all possible combinations are tested

Helps identify missing requirements or logic gaps

Provides clear documentation of complex business rules

Example : Online shopping discount system

Conditions: Customer type (Regular/Premium), Purchase amount (>$100 / ≤$100), Promo code (Valid/Invalid)

Actions: Apply 10% discount, Apply 20% discount, No discount, Add free shipping

Decision Table Testing Example

In this table, there are

2 conditions : UserID, Password

2 Actions : Login Successfully,

Error showing 'Invalid Credentials'

3 Options : Blank, Valid, Invalid.

	Rule1	Rule2	Rule3	Rule4	Rule5	Rule6	Rule7	Rule8	Rule9
Conditions	TC01	TC02	TC03	TC04	TC05	TC06	TC07	TC08	TC09
UserID	Blank	Valid	Invalid	Blank	Blank	Invalid	Valid	Invalid	Valid
Password	Blank	Blank	Blank	Valid	Invalid	Valid	Invalid	Invalid	Valid
Actions									
Login Successfully									Execute
Error showing 'Invalid Credentials'	Execute	Execute	Execute	Execute	Execute	Execute	Execute	Execute	

So, there are total **9** of test cases: 3^2 = 9. Then reduce the number of test cases.

Rules 1,2, 3, 4, and 5 cover the same action Item "Invalid Credentials" with options Blank and Invalid. Hence, we can consider any one of these test cases TC01 OR TC02 OR TC03 OR TC04 OR TC05

Rules 6,7, and 8 cover the same action Item "Invalid Credentials" with options Valid and Invalid. Hence, we can consider any of these test cases TC06 OR TC07 OR TC08

Rule 9 covers the action item "Login Successfully" with all

valid options. Hence, we should consider the test case TC09.

Finally, we will collapse the Decision Table as shown:

	Rule2	Rule8	Rule9
Conditions	TC02	TC08	TC09
UserID	Valid	Invalid	Valid
Password	Blank	Invalid	Valid
Actions			
Login Successfully			Execute
Error showing 'Invalid Credentials'	Execute	Execute	

SWINBURNE UNIVERSITY OF TECHNOLOGY

State Transition Testing

Definition :

Focuses on the behaviour of a system as it transitions between different states.

Key concepts:

State: A condition or situation of the system at a specific time

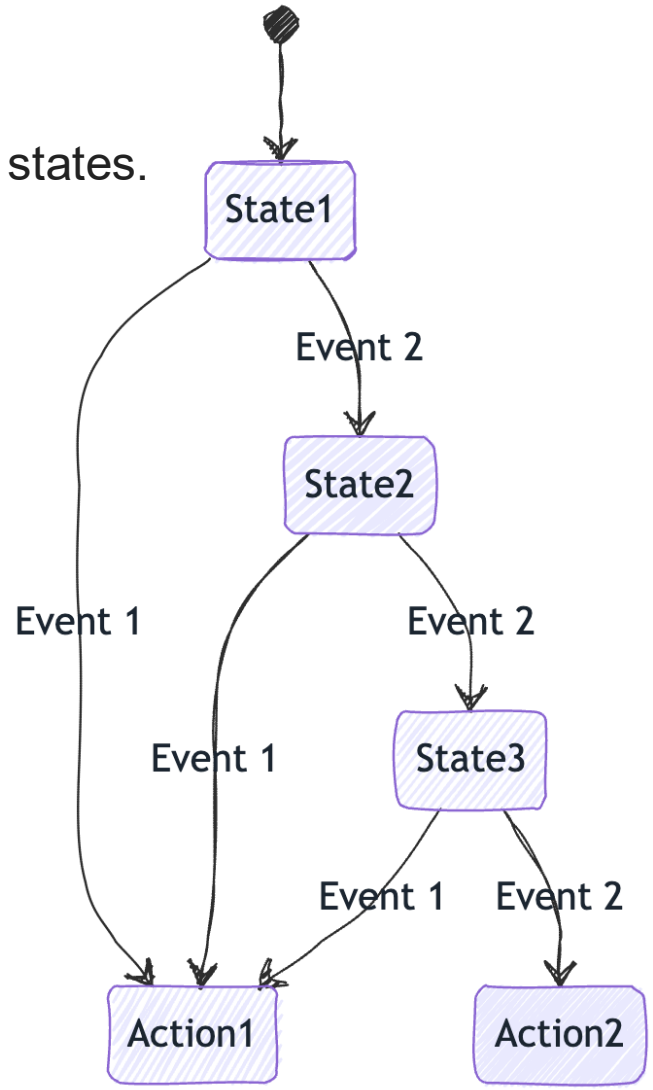
Transition: A change from one state to another

Event: A trigger that causes a state transition

Action: An activity that occurs during a state transition

Steps to apply :

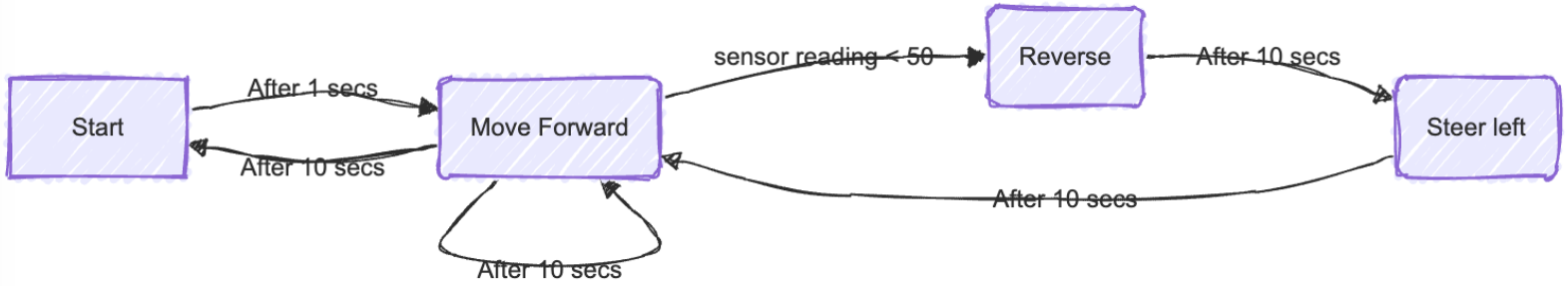
1. Identify all possible states of the system
2. Determine events that trigger state transitions
3. Create a state transition diagram or table
4. Design test cases to cover all state transitions
5. Include both valid and invalid state transitions



State Transition Testing

Types of state transition models :

1. Finite State Machine (FSM)
2. Harel Statechart
3. UML State Machine Diagram



Coverage criteria:

0-switch coverage: Test all states (but not the transitions between them).

Ensures every state is visited at least once.

1-switch coverage: Test all transitions.

Ensures every transition from one state to another is executed at least once.

n-switch coverage: Test sequences of n transitions.

Ensures that all possible sequences of n consecutive transitions are covered.

Benefits :

Effective for testing systems with distinct states (e.g., ATMs, vending machines)

Helps identify missing or incorrect transitions

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Pairwise Testing (All-Pairs Testing)

Definition :

A combinatorial testing technique that tests all possible discrete combinations of each pair of input parameters

Principle:

Most defects are caused by interactions between two parameters

Steps to apply:

1. Identify input parameters and their possible values. Generate all possible pairs of parameter values
2. Create test cases covering all pairs. Use pairwise testing tools for efficient test case generation

Benefits:

Significantly reduces the number of test cases compared to exhaustive testing

Provides good defect detection capability

Effective for testing configurations and input combinations

Example : Testing a web application across different platforms

Parameters: Browser (Chrome, Safari), OS (Windows, macOS, Linux), Screen Resolution (1080p, 4K)

Pairwise combinations ensure all pairs of these parameters are tested

Error Guessing

Definition :

A test design technique where the experience of the tester is used to anticipate potential defects

Principle:

Based on the tester's intuition, knowledge of the system, and past experience with similar applications

Steps to apply:

1. Analyze the system requirements and design. Identify potential weak points or error-prone areas
2. Create a list of possible error scenarios. Design test cases based on these scenarios

Benefits:

Maintain a checklist of common errors for reference

Encourage brainstorming sessions among testers

Combine with exploratory testing for better results

Limitations:

Not repeatable or systematic

Effectiveness varies based on tester's experience

May lead to biased testing if used exclusively

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Exploratory Testing

Definition :

Simultaneous learning, test design, and test execution

Characteristics:

Emphasizes personal freedom and responsibility of the tester

Guided by a mission, but open to opportunity and discovery

Iterative and adaptive process

Benefits:

Rapid defect discovery, especially for new or changing systems

Leverages tester's creativity and intuition

Effective for systems with poor or outdated documentation

Helps in understanding the system from a user's perspective

Challenges:

Difficult to plan and estimate effort. Difficult to reproduce defects

May miss systematic coverage of all features

Requires skilled and experienced testers

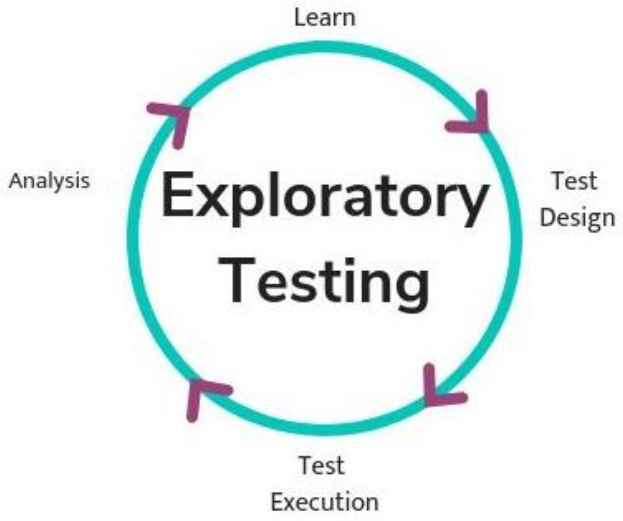


Image from [Pinterest]

- Introduction to Software Testing
- Types of Software Testing
- Test Planning and Strategy
- Test Case Design Techniques
- **Test Execution and Reporting**
- Automated Testing

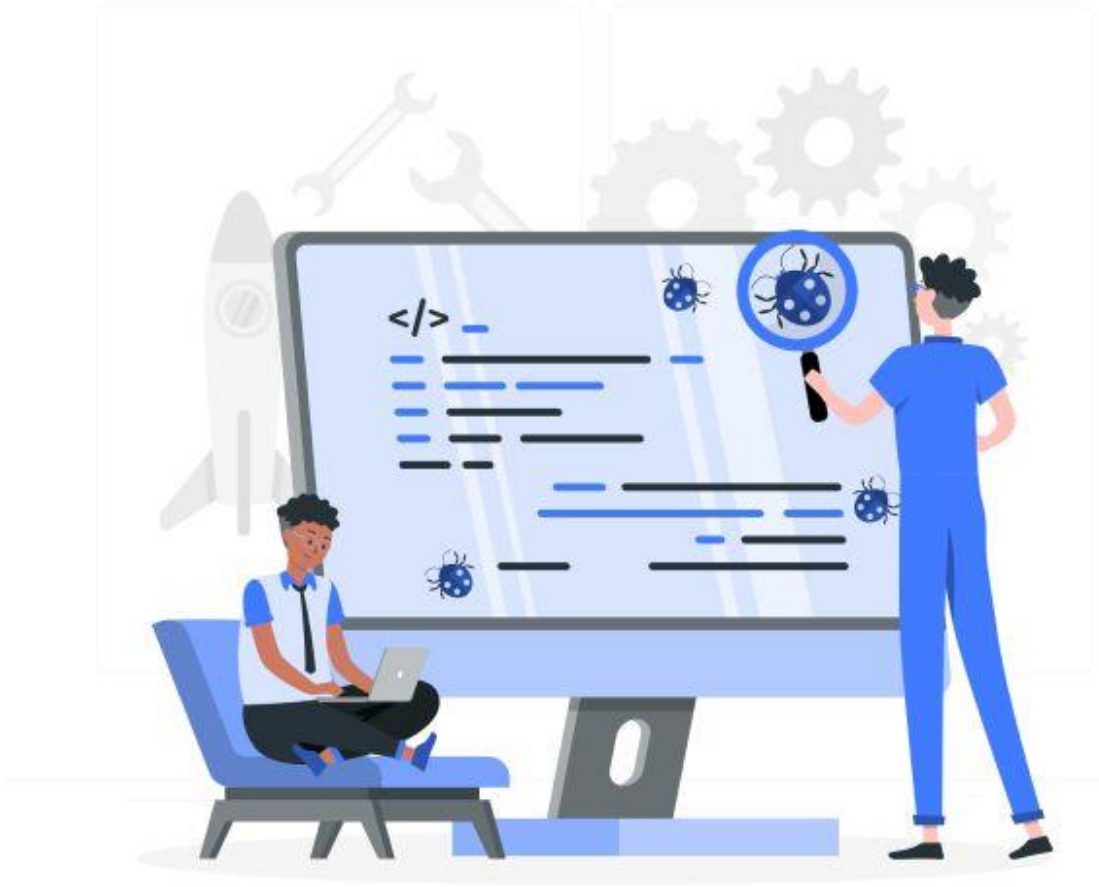


Image from [Pinterest]

Test Execution Process

Definition :

The process of running tests on a software system to evaluate its behavior and identify defects

Phases of test execution:

1. Test environment setup
2. Test data preparation
3. Test case execution
4. Result analysis and defect reporting
5. Regression testing
6. Test closure

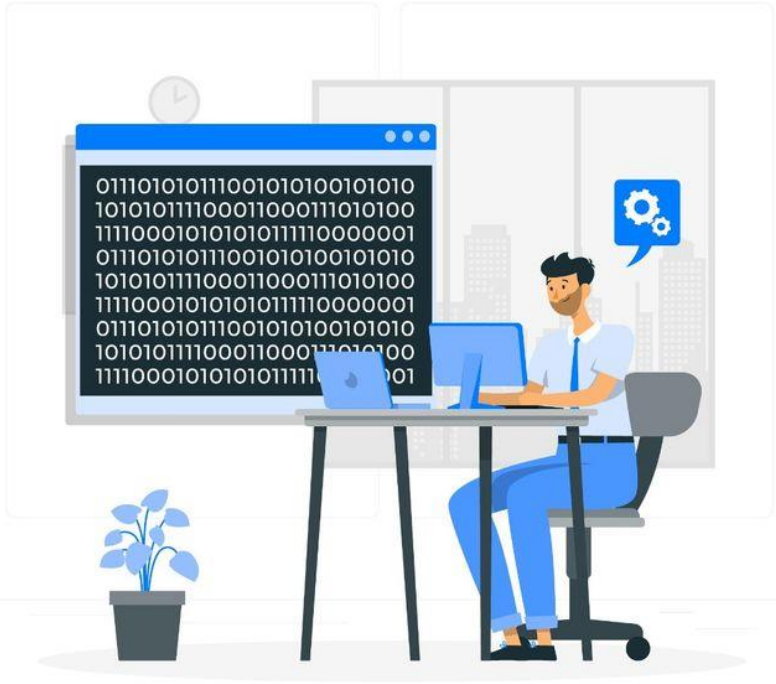


Image from [Pinterest]

Best practices for test execution:

Follow a structured approach outlined in the test plan

Maintain detailed logs of all testing activities

Ensure consistency in the test environment across executions

Prioritize test cases based on risk and importance

Use a mix of manual and automated testing for optimal coverage

Regularly update test cases to reflect system changes

Detailed breakdown of Each Phase in Test Execution

1. **Test environment setup:**
   - Hardware configuration
   - Software installation and configuration
   - Network setup (if applicable)
   - Database initialization
   - Version control system integration
2. **Test data preparation:**
   - Creating test data sets
   - Data anonymization for sensitive information
   - Database seeding
   - Mock data generation for edge cases
3. **Test case execution:**
   - Manual execution:
     - Following step-by-step instructions
     - Recording actual results
     - Comparing with expected results
   - Automated execution:
     - Running automated test scripts
     - Parallel execution for faster results
     - Continuous integration/continuous deployment (CI/CD) pipeline integration
4. **Result analysis and defect reporting:**
   - Analyzing test results
   - Identifying and categorizing defects
   - Prioritizing defects based on severity and impact
   - Creating detailed defect reports
5. **Regression testing:**
   - Re-running affected test cases after fixes
   - Ensuring new changes don't introduce new bugs
   - Automated regression test suite maintenance
6. **Test closure:**
   - Evaluating exit criteria
   - Preparing test summary reports
   - Conducting retrospective meetings
   - Archiving test artifacts

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Defect Management Process

Definition :

The systematic approach to identifying, tracking, and resolving software defects throughout the development lifecycle

Defect lifecycle stages:

1. New: Defect is identified and reported
2. Assigned: Defect is allocated to a developer for investigation
3. Open: Developer acknowledges and starts working on the defect
4. Fixed: Developer implements a solution for the defect
5. Ready for Test: Fixed defect is ready for retesting
6. Retest: Tester verifies the fix
7. Closed: Defect is confirmed as resolved
8. Reopened: Defect reoccurs or fix is insufficient

Severity levels:

1. Critical: System crash, data loss, security breach
2. High: Major functionality is impacted, no workaround available
3. Medium: Functionality is impacted but has a workaround
4. Low: Minor issues, cosmetic problems

Priority levels:

1. Urgent: Requires immediate attention
2. High: Should be fixed in the next build
3. Medium: Should be fixed in a future release
4. Low: Can be fixed when time permits

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Test Reporting

Types of Test Reports :

Test Execution Summary:

a) Overview of test execution status

b) Number of tests (passed, failed, blocked)

c) Execution progress over time

Defect Summary:

a) Total number of defects

b) Defects by severity and status

c) Trend analysis of defects over time

Coverage Report:

a) Requirements coverage

b) Feature coverage

c) Risk coverage (for risk-based testing)

Components of a Comprehensive Test Report

Executive Summary:

a) Overall status and key findings

b) Major achievements and challenges

c) Recommendations

Detailed Results:

a) Test case execution details

b) Defect analysis

c) Coverage analysis

Metrics and KPIs:

a) Test effectiveness

b) Defect density

c) Test execution productivity

SWINBURNE UNIVERSITY OF TECHNOLOGY

- Introduction to Software Testing
- Types of Software Testing
- Test Planning and Strategy
- Test Case Design Techniques
- Test Execution and Reporting
- **Automated Testing**

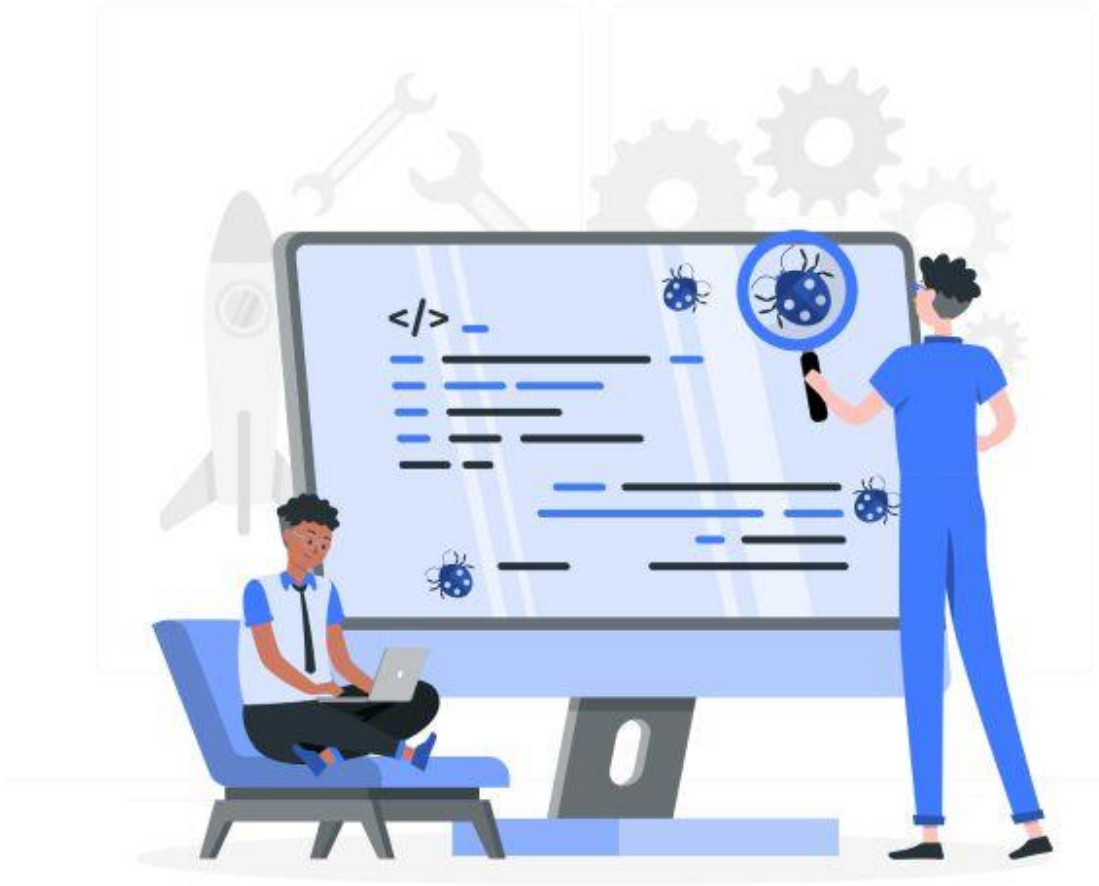


Image from [Pinterest]

Introduction to Automated Testing

Definition :

Automated testing is the use of special software tools to execute pre-scripted tests on a software application automatically

It involves the automation of manual testing processes, including test case creation, execution, and results analysis

Key Characteristics:

Repeatable and consistent test execution

Ability to run tests 24/7 without human intervention

Support for regression testing and continuous integration

Capability to simulate thousands of virtual users



Image from [Pinterest]

Advantages:

Increased test coverage and depth

Faster execution of large test suites

Reduction in human errors and bias

Cost-effective for long-term projects and frequently changing applications

Challenges:

High initial investment in tools and training

Maintenance of test scripts as application evolves

Test Automation Frameworks

Data-Driven Framework :

Definition: Separates test data from test scripts

Characteristics: a) Test data stored in external files (e.g., Excel, CSV) b) Single test script can run multiple test cases

Advantages: a) Easy to add new test cases without modifying scripts b) Improves test maintainability

Challenges: a) Requires careful data management b) May not be suitable for all types of tests

Keyword-Driven Framework :

Definition: Uses keywords to represent actions in the application

Characteristics: a) Test cases written in a tabular format with keywords b) Keyword implementations separate from cases

Advantages: a) Non-technical testers can create test cases b) High reusability of code

Challenges: a) Initial setup time can be significant b) Requires maintenance of keyword library

Hybrid Framework :

Definition: Combines elements of multiple framework types

Characteristics: a) Customized to fit specific project needs b) Often integrates data-driven and keyword-driven approaches

Advantages: a) Flexibility to handle various testing requirements b) Can leverage strengths of different frameworks

Challenges: a) Complexity in design and implementation b) Requires experienced automation engineers

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Popular Automation Tools

1. **Selenium**

   Supports multiple browsers and programming languages.

   Can be integrated with various testing frameworks (e.g., TestNG, JUnit).

   Pros: Wide community support, flexibility in test script creation.

   Cons: Steep learning curve, lacks built-in reporting.



Image from [Pinterest]

2. **Appium**

   Cross-platform tool for Android and iOS testing.

   Supports native, hybrid, and web apps.

   Pros: Single API for both platforms.

   Cons: Complex setup, slower execution compared to native tools.



Image from [medium.com]



Image from [kartaca.com]

3. **JMeter**

   Performance and load testing tool supporting various protocols (HTTP, JDBC, etc.).

   Pros: Powerful analysis and reporting, extendable with plugins.

   Cons: Limited client-side performance testing, resource-intensive for large tests.

Thank you



Image from [Pinterest]

SWINBURNE UNIVERSITY OF TECHNOLOGY